

INFORME · TCC · BACKEND

Escrevendo Bons Testes Unitários

Um guia prático para a equipe — com exemplos reais do nosso Domain e os erros mais comuns para evitar.

O QUE É UM TESTE UNITÁRIO

Um teste unitário verifica **um único comportamento** de uma única classe, sem banco de dados, sem rede, sem nada externo. No nosso caso, o Domain é C# puro — sem EF, sem Argon2, sem nada — então cada classe é trivial de testar: instancia, chama o método, confere o resultado.

REGRA 01

Um teste = um comportamento

Não teste três coisas no mesmo **[Fact]**. Se falhar, você não sabe qual dos três quebrou.

REGRA 02

Sem lógica no teste

Nenhum **if**, nenhum **for**. Se precisar de lógica, use **[Theory]** com **[InlineData]**.

REGRA 03

O nome documenta o comportamento

MetodoTestado_Cenario_ResultadoEsperado.
Quem lê o nome deve entender o que o teste verifica sem abrir o código.

REGRA 04

Arrange → Act → Assert

Prepare os dados, execute a ação, verifique o resultado. Sempre nessa ordem, sempre separado.

REGRA 05

REGRA 06

Testes devem ser determinísticos

Teste o comportamento, não a implementação

Não teste se `_lista.Count == 1` internamente. Teste se o método retorna o que deveria retornar.

Um teste que passa às vezes e falha outras é pior do que nenhum teste. Não dependa de horário, aleatoriedade ou ordem de execução.

ANATOMIA DE UM TESTE BEM FEITO

```
[Fact]
public void UpdateUsername_ComNomeValido_AtualizaOUsername

    // Arrange - prepara o estado inicial
    var      new User "gabriel" "gabriel@etec.sp.gov.br" "hash" UserRole

    // Act - executa o comportamento que está sendo testado
    UpdateUsername "gabrielv2"

    // Assert - verifica o resultado. Um único Assert por Fact.
    Assert Equal "gabrielv2"
```

O comentário `// Arrange / Act / Assert` é opcional depois que o padrão for natural para o time, mas ajuda enquanto estão aprendendo.

[FACT] VS [THEORY]

[Fact] — um único cenário

Use quando o comportamento só precisa ser verificado uma vez, com valores fixos.

```
[Fact]
public void Construtor_DeveIniciarComoAtiva
```

```
var session = new Session("192.168.0.10", DateTimeOffset.UtcNow, "hash", Guid.NewGuid());  
  
Assert.True(session.IsValid);
```

[Theory] + [InlineData] — múltiplos cenários, mesma lógica

Use quando você quer testar o mesmo comportamento com entradas diferentes. O xUnit roda o método uma vez por `[InlineData]`.

```
[Theory]  
[InlineData("gabriel123", true)] // válido  
[InlineData("", false)] // vazio - inválido  
[InlineData("com espaço", false)] // espaço - inválido  
[InlineData("aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa", false)]  
public void ValidateUsername_AceitaOuRejeita(string username, bool expected)  
{  
    if (expected)  
        Assert.True(User.ValidateUsername(username));  
    else  
        Assert.Throws<ArgumentException>(() => User.ValidateUsername(username));  
}
```

Cada `[InlineData]` aparece como um teste separado no runner — se um falhar, os outros ainda rodam.

TESTANDO EXCEÇÕES COM ASSERT.THROWS

Quando um método deve lançar exceção em entradas inválidas, use `Assert.Throws`. Não use try/catch nos testes.

ERRADO

CERTO

```
// Não faça isso - esconde a exceção
try
    User ValidateUsername ""
    Assert Fail "devia ter lançado"
catch ArgumentException
    // ok?
```

```
// Assert.Throws: limpo, idiomático
Assert Throws ArgumentException
    User ValidateUsername ""
```

TESTANDO TIMESTAMPS COM ASSERT.INRANGE

Nunca compare `DateTimeOffset` com `Assert.Equal` — vai falhar por diferença de microssegundos. Capture o intervalo antes e depois da criação.

```
[Fact]
public void Construtor_LoginAtDeveSerHoraAtual

    var        DateTimeOffset

    var        new Session "192.168.0.10" DateTimeOffset "hash" Guid NewGuid

    var        DateTimeOffset

    // LoginAt deve estar dentro da janela de execução do teste
    Assert InRange
```

O QUE NÃO VALE A PENA TESTAR

- ✗ Getters e setters automáticos — `{ get; set; }` são do compilador, não da lógica de negócio
- ✗ Construtores sem lógica — se só atribui propriedades sem validação, não há o que testar

- ✗ Código do EF Core / banco de dados — isso é teste de integração, não unitário
- ✗ Código de terceiros (Argon2, Regex da stdlib) — confie na lib
- ✓ Validações com regras de negócio — tamanho máximo, formato, valores proibidos
- ✓ Mutações de estado — métodos que alteram propriedades da entidade
- ✓ Casos de borda — string vazia, Guid.Empty, data no passado
- ✓ Exceções esperadas — entradas que devem lançar `ArgumentException`

ERROS ENCONTRADOS NOS TESTES DESSA SEMANA

Import não usado

O `UserTests.cs` importava `System.Security.Cryptography.X509Certificates` e `Xunit.Abstractions` sem usar nenhum dos dois. Imports desnecessários são ruído — remova sempre.

Teste que não compila depois de mudar a assinatura

O construtor de `Session` foi alterado para remover `expiresAt` (que passou a ser calculado internamente), mas os testes continuaram passando o parâmetro. **Quando você altera a assinatura de um construtor ou método, rode os testes logo depois.** Esse erro dura horas se ninguém rodar o CI.

```
// Antes (quebrado): expiresAt não existe mais no construtor
var          new Session          DateTimeOffset          7
```

```
// Depois (correto): construtor simplificado  
var new Session "192.168.0.10" DateTimeOffset "hash"
```

Variáveis mortas no teste

O `GroupTests.cs` declarava `var date = DateTime.Now` e `var dateAfter = DateTime.Now` mas nunca usava nenhuma das duas. Variável declarada e não usada = intenção incompleta. Ou você esqueceu de escrever o Assert, ou o código pode ser deletado.

Arquivo de teste vazio com nome errado

O `AuditLogTests.cs` continha uma classe `internal class DatetimeTests` sem nenhum teste. Arquivo sem teste não gera falha, mas polui o projeto e confunde quem lê. Se não tem o que testar ainda, deixe um placeholder real com comentários `// TODO`.

PRÓXIMOS TESTES A ESCREVER

Cada `// TODO` que está nos arquivos corresponde a um método que ainda não existe no Domain. Quando o método for implementado, o teste correspondente deve ser desbloqueado na mesma PR.

☐ `Session.Revoke()` → `IsActive` deve virar `false`

☐ `Session.IsExpired()` → com `ExpiresAt` no passado deve retornar `true`

☐ `Device.MarkOnline()` / `MarkOffline()` → Status deve mudar

☐ `Group.Deactivate()` → `IsActive` deve virar `false`

■ `EnrollmentToken.Redeem()` → `IsUsed` deve virar `true`, não deve permitir reuso